

C Aptitude Questions with Answers

What is the output of following C programs?

```
1)
main()
{
clrscr();
}
clrscr();
```

Answer:

No output/error

Explanation:

The first clrscr() occurs inside a function. So it becomes a function call. In the second clrscr(); is a function declaration (because it is not inside any function).

```
2)
enum colors {BLACK,BLUE,GREEN}
main()
{

printf(“%d..%d..%d”,BLACK,BLUE,GREEN);

return(1);
}
```

Answer:

0..1..2

Explanation:

enum assigns numbers starting from 0, if not explicitly defined.

```
3)
void main()
{
char far *farther,*farthest;

printf(“%d..%d”,sizeof(farther),sizeof(farthest));

}
```

Answer:

4..2

Explanation:

the second pointer is of char type and not a far pointer

```
4)
main()
{
int i=400,j=300;
printf(“%d..%d”);
```

```
}
```

Answer:

400..300

Explanation:

printf takes the values of the first two assignments of the program. Any number of printf's may be given. All of them take only the first two values. If more number of assignments given in the program, then printf will take garbage values.

5)

```
main()
```

```
{
```

```
char *p;
```

```
p="Hello";
```

```
printf("%c\n",&*p);
```

```
}
```

Answer:

H

Explanation:

* is a dereference operator & is a reference operator. They can be applied any number of times provided it is meaningful. Here p points to the first character in the string "Hello". *p dereferences it and so its value is H. Again & references it to an address and * dereferences it to the value H.

6)

```
main()
```

```
{
```

```
int i=1;
```

```
while (i<=5)
```

```
{
```

```
printf("%d",i);
```

```
if (i>2)
```

```
goto here;
```

```
i++;
```

```
}
```

```
}
```

```
fun()
```

```
{
```

```
here:
```

```
printf("PP");
```

```
}
```

Answer:

Compiler error: Undefined label 'here' in function main

Explanation:

Labels have functions scope, in other words The scope of the labels is limited to functions . The label 'here' is available in function fun() Hence it is not visible in function main.

7)

```
main()
```

```
{
```

```
int i=0;
```

```
for(;i++;printf("%d",i)) ;
printf("%d",i);
}
```

Answer:

1

Explanation:

before entering into the for loop the checking condition is “evaluated”. Here it evaluates to 0 (false) and comes out of the loop, and i is incremented (note the semicolon after the for loop).

```
#include<stdio.h>
main()
{
char s[]={ 'a','b','c','\n','c','\0'};
char *p,*str,*str1;
p=&s[3];
str=p;
str1=s;
printf("%d",++*p + ++*str1-32);
}
```

Answer:

M

Explanation:

p is pointing to character ‘\n’.str1 is pointing to character ‘a’ ++*p meAnswer:”p is pointing to ‘\n’ and that is incremented by one.” the ASCII value of ‘\n’ is 10. then it is incremented to 11. the value of ++*p is 11. ++*str1 meAnswer:”str1 is pointing to ‘a’ that is incremented by 1 and it becomes ‘b’. ASCII value of ‘b’ is 98. both 11 and 98 is added and result is subtracted from 32.

i.e. $(11+98-32)=77$ (“M”);

```
9)
#include<stdio.h>
main()
{
struct xx
{
int x=3;
char name[]="hello";
};
struct xx *s=malloc(sizeof(struct xx));
printf("%d",s->x);
printf("%s",s->name);
}
```

Answer:

Compiler Error

Explanation:

Initialization should not be done for structure members inside the structure declaration

```

10)
#include<stdio.h>
main()
{
struct xx
{
int x;
struct yy
{
char s;
struct xx *p;
};
struct yy *q;
};
}

```

Answer:

Compiler Error

Explanation:

in the end of nested structure yy a member have to be declared.

```

11)
main()
{
extern int i;
i=20;
printf(“%d”,sizeof(i));
}

```

Answer:

Linker error: undefined symbol ‘_i’.

Explanation:

extern declaration specifies that the variable i is defined somewhere else. The compiler passes the external variable to be resolved by the linker. So compiler doesn’t find an error. During linking the linker searches for the definition of i. Since it is not found the linker flags an error.

```

12)
main()
{
printf(“%d”, out);
}
int out=100;

```

Answer:

Compiler error: undefined symbol out in function main.

Explanation:

The rule is that a variable is available for use from the point of declaration. Even though a is a global variable, it is not available for main. Hence an error.

```

13)
main()
{
extern out;

```

```
printf("%d", out);  
}
```

int out=100;

Answer:

100

Explanation:

This is the correct way of writing the previous program.

14)

```
main()  
{
```

```
show();  
}
```

```
void show()  
{
```

```
printf("I'm the greatest");  
}
```

Answer:

Compiler error: Type mismatch in redeclaration of show.

Explanation:

When the compiler sees the function show it doesn't know anything about it. So the default return type (ie, int) is assumed. But when compiler sees the actual definition of show mismatch occurs since it is declared as void. Hence the error.

The solutions are as follows:

1. declare void show() in main() .
2. define show() before main().
3. declare extern void show() before the use of show().

15)

```
main()  
{
```

```
static char names[5][20]={ "pascal", "ada", "cobol", "fortran", "perl" };  
int i;
```

```
char *t;
```

```
t=names[3];
```

```
names[3]=names[4];
```

```
names[4]=t;
```

```
for (i=0;i<=4;i++)
```

```
printf("%s",names[i]);
```

```
}
```

Answer:

Compiler error: Lvalue required in function main

Explanation:

Array names are pointer constants. So it cannot be modified.

16)

```
void main()  
{
```

```
{
```

```
int i=5;
```

```
printf("%d",i++ + ++i);
```

```
}
```

Answer:

Output Cannot be predicted exactly.

Explanation:

Side effects are involved in the evaluation of `i`

17)

```
main()
```

```
{
```

```
char string[]="Hello World";
```

```
display(string);
```

```
}
```

```
void display(char *string)
```

```
{
```

```
printf("%s",string);
```

```
}
```

Answer:

Compiler Error : Type mismatch in redeclaration of function display

Explanation :

In third line, when the function display is encountered, the compiler doesn't know anything about the function display. It assumes the arguments and return types to be integers, (which is the default type). When it sees the actual function display, the arguments and type contradicts with what it has assumed previously. Hence a compile time error occurs.

18)

```
main()
```

```
{
```

```
int c=- -2;
```

```
printf("c=%d",c);
```

```
}
```

Answer:

```
c=2;
```

Explanation:

Here unary minus (or negation) operator is used twice. Same maths rules applies, ie. minus * minus= plus.

Note:

However you cannot give like `-2`. Because `—` operator can only be applied to variables as a decrement operator (eg., `i--`). `2` is a constant and not a variable.

19)

```
#define int char
```

```
main()
```

```
{
```

```
int i=65;
```

```
printf("sizeof(i)=%d",sizeof(i));
```

```
}
```

Answer:

```
sizeof(i)=1
```

Explanation:

Since the `#define` replaces the string `int` by the macro `char`

20)

```
main()
{
int i=10;
i=!i>14;
Printf ("i=%d",i);
}
Answer:
i=0
```

Explanation:

In the expression `!i>14`, NOT (!) operator has more precedence than '`>`' symbol. `!` is a unary logical operator. `!i` (`!10`) is 0 (not of true is false). `0>14` is false (zero).

21)

```
#include<stdio.h>
main()
{
char s[]={ 'a','b','c','\n','c','\0'};
char *p,*str,*str1;
p=&s[3];
str=p;
str1=s;
printf("%d",++*p + ++*str1-32);
}
Answer:
77
```

Explanation:

`p` is pointing to character '`\n`'. `str1` is pointing to character '`a`'. `++*p`. "`p` is pointing to '`\n`' and that is incremented by one." the ASCII value of '`\n`' is 10, which is then incremented to 11. The value of `++*p` is 11. `++*str1`, `str1` is pointing to '`a`' that is incremented by 1 and it becomes '`b`'. ASCII value of '`b`' is 98. Now performing $(11 + 98 - 32)$, we get 77 ("M"); So we get the output 77 :: "M" (Ascii is 77).

22)

```
#include<stdio.h>
main()
{
int a[2][2][2] = { {10,2,3,4}, {5,6,7,8} };
int *p,*q;
p=&a[2][2][2];
*q=***a;
printf("%d—%d",*p,*q);
}
Answer:
SomeGarbageValue—1
```

Explanation:

`p=&a[2][2][2]` you declare only two 2D arrays, but you are trying to access the third 2D(which you are not declared) it will print garbage values. `*q=***a` starting address of `a` is

assigned integer pointer. Now q is pointing to starting address of a. If you print *q, it will print first element of 3D array.

23)

```
#include<stdio.h>
main()
{
struct xx
{
int x=3;
char name[]="hello";
};
struct xx *s;
printf("%d",s->x);
printf("%s",s->name);
}
```

Answer:

Compiler Error

Explanation:

You should not initialize variables in declaration

24)

```
#include<stdio.h>
main()
{
struct xx
{
int x;
struct yy
{
char s;
struct xx *p;
};
struct yy *q;
};
}
```

Answer:

Compiler Error

Explanation:

The structure yy is nested within structure xx. Hence, the elements are of yy are to be accessed through the instance of structure xx, which needs an instance of yy to be known. If the instance is created after defining the structure the compiler will not know about the instance relative to xx. Hence for nested structure yy you have to declare member.

25)

```
main()
{
printf("\nab");
printf("\bsi");
printf("\rha");
}
```


}

Answer:

hai

Explanation:

\n – newline

\b – backspace

\r – linefeed